Pimpri-Chinchwad Education Trust's
Pimpri-Chinchwad College of Engineering
(An Autonomous Institute)
Affiliated to Savitribai Phule Pune University (SPPU) ISO 21001:2018 Certified by TUV

DEPARTMENT OF INFORMATION TECHNOLOGY

Assignment No. 3

Subject	DBMS Lab
Assignment No.	3
Topic	All types of Join, Sub-Query and View
Student Name	Madhav Khobare
Roll Number	124B1F023
Date	24/02/2026
Div & Batch	SY IT A1
Guided by: -	Ms. Alpana A. Borse

Academic Year: 2025-26 Semester: IV

Aim:

To study and implement different types of SQL queries using joins, subqueries, views, and GROUP BY with HAVING clause.

Objectives:

- To understand different types of joins (INNER, LEFT, RIGHT).
- To implement subqueries and nested queries.
- To create and use views in SQL.
- To perform grouping of data using GROUP BY clause.
- To filter grouped data using HAVING clause

Case Study: Airline Management System

SQL Queries:

Part A: Joins, Subqueries, and Views

Theory: Joins, Subqueries and Views

Joins are used to combine data from multiple tables based on related columns. Different types of joins such as INNER JOIN, LEFT JOIN, and RIGHT JOIN are used depending on the requirement.

Subqueries are queries written inside another query and are used to perform complex filtering and data retrieval.

Views are virtual tables created using SQL queries, which help in simplifying complex queries and improving security.

1. Inner Join:

Get passenger names along with their flight names and seat numbers.

```
mysql> SELECT p.pname, f.flight_name, b.seat_no
    -> FROM passenger p
    -> INNER JOIN booking b ON p.passenger_id = b.passenger_id
    -> INNER JOIN flight f ON b.flight_id = f.flight_id;
+-------------------+-------------+---------+
| pname             | flight_name | seat_no |
+-------------------+-------------+---------+
| Rahul Sharma      | AI101       | 12A     |
| Neha Verma        | IN202       | 14B     |
| Sneha Kulkarni    | EM404       | 8D      |
+-------------------+-------------+---------+
3 rows in set (0.00 sec)
```

---

2. Left Join:

Display all passengers and their booking details.

```
mysql> SELECT p.pname, b.booking_id, b.seat_no
    -> FROM passenger p
    -> LEFT JOIN booking b ON p.passenger_id = b.passenger_id;
+-------------------+------------+---------+
| pname             | booking_id | seat_no |
+-------------------+------------+---------+
| Rahul Sharma      | 1          | 12A     |
| Neha Verma        | 2          | 14B     |
| Amit Patil        | NULL       | NULL    |
| Sneha Kulkarni    | 4          | 8D      |
+-------------------+------------+---------+
4 rows in set (0.00 sec)
```

---

3. Right Join:

List all flights and passengers who booked them.

```
mysql> SELECT f.flight_name, f.source, f.destination, p.pname
    -> FROM booking b
    -> RIGHT JOIN flight f ON b.flight_id = f.flight_id
    -> LEFT JOIN passenger p ON b.passenger_id = p.passenger_id;
+-------------+--------+-------------+-------------------+
| flight_name | source | destination | pname             |
+-------------+--------+-------------+-------------------+
| AI101       | Mumbai | Delhi       | Rahul Sharma      |
| IN202       | Pune   | Bangalore   | Neha Verma        |
| SJ303       | Delhi  | Goa         | NULL              |
| EM404       | Dubai  | Mumbai      | Sneha Kulkarni    |
+-------------+--------+-------------+-------------------+
4 rows in set (0.00 sec)
```

---

4. Subquery (WHERE):

Show flights operated by Indian airlines.

```
mysql> SELECT flight_name, source, destination, price
    -> FROM flight
    -> WHERE airline_id IN
    -> (SELECT airline_id FROM airline WHERE country = 'India');
+-------------+--------+-------------+--------+
| flight_name | source | destination | price  |
+-------------+--------+-------------+--------+
| AI101       | Mumbai | Delhi       | 5500.0 |
| IN202       | Pune   | Bangalore   | 4500.0 |
| SJ303       | Delhi  | Goa         | 6000.0 |
+-------------+--------+-------------+--------+
3 rows in set (0.00 sec)
```

5. Nested Subquery:

Find passengers who booked flights costing more than 5000.

```
mysql> SELECT pname
    -> FROM passenger
    -> WHERE passenger_id IN (
    -> SELECT passenger_id
    -> FROM booking
    -> WHERE flight_id IN (
    -> SELECT flight_id FROM flight WHERE price > 5000));
+-------------------+
| pname             |
+-------------------+
| Rahul Sharma      |
| Sneha Kulkarni    |
+-------------------+
2 rows in set (0.00 sec)
```

---

6. Scalar Subquery:

Find passenger(s) who booked the highest priced flight.

```
mysql> SELECT p.pname, f.flight_name, f.price
    -> FROM passenger p
    -> JOIN booking b ON p.passenger_id = b.passenger_id
    -> JOIN flight f ON b.flight_id = f.flight_id
    -> WHERE f.price = (SELECT MAX(price) FROM flight);
+-------------------+-------------+---------+
| pname             | flight_name | price   |
+-------------------+-------------+---------+
| Sneha Kulkarni    | EM404       | 15000   |
+-------------------+-------------+---------+
1 row in set (0.00 sec)
```

---

7. Create View:

Create a view for booking itinerary.

```
mysql> CREATE VIEW BookingItinerary AS
    -> SELECT b.booking_id, p.pname, f.flight_name, f.source, f.destination
    -> FROM booking b
    -> JOIN passenger p ON b.passenger_id = p.passenger_id
    -> JOIN flight f ON b.flight_id = f.flight_id;
Query OK, 0 rows affected (0.02 sec)
```

---

8. Query View:

Display all bookings from Mumbai.

```
mysql> SELECT * FROM BookingItinerary
    -> WHERE source = 'Mumbai' OR destination = 'Mumbai';
```

```
+------------+------------------+-------------+--------+-------------+
| booking_id | pname            | flight_name | source | destination |
+------------+------------------+-------------+--------+-------------+
| 1          | Rahul Sharma     | AI101       | Mumbai | Delhi       |
| 4          | Sneha Kulkarni   | EM404       | Dubai  | Mumbai      |
+------------+------------------+-------------+--------+-------------+
2 rows in set (0.00 sec)
```

Part B: GROUP BY and HAVING

Theory: GROUP BY and HAVING

The GROUP BY clause is used to group rows having the same values in specified columns into summary rows. It is often used with aggregate functions like COUNT, AVG, SUM, etc.

The HAVING clause is used to filter grouped data, unlike the WHERE clause which filters rows before grouping.

9. GROUP BY:

Count total flights per airline.

```
mysql> SELECT a.airline_name, COUNT(f.flight_id) AS total_flights
    -> FROM airline a
    -> LEFT JOIN flight f ON a.airline_id = f.airline_id
    -> GROUP BY a.airline_name;
+--------------+---------------+
| airline_name | total_flights |
+--------------+---------------+
| Air India    | 1             |
| Indigo       | 1             |
| Emirates     | 1             |
+--------------+---------------+
3 rows in set (0.00 sec)
```

---

10. GROUP BY with HAVING:

Find airlines with average price > 5000.

```
mysql> SELECT a.airline_name, AVG(f.price) AS avg_price
    -> FROM airline a
    -> JOIN flight f ON a.airline_id = f.airline_id
    -> GROUP BY a.airline_name
    -> HAVING AVG(f.price) > 5000;
+--------------+-----------+
| airline_name | avg_price |
+--------------+-----------+
| Air India    | 5500      |
| Emirates     | 15000     |
+--------------+-----------+
2 rows in set (0.00 sec)
```

Case Study: Civic Voice Interface System

SQL Queries:

Part A: Joins, Subqueries, and Views

Theory: Joins, Subqueries and Views

Joins help in retrieving combined data from multiple related tables. Subqueries allow performing complex queries within another query. Views provide a simplified and reusable way to access data.

1. Inner Join

```
mysql> SELECT u.name, c.state, c.district
    -> FROM user u
    -> INNER JOIN citizenprofile c ON u.user_id = c.user_id;
+--------------+-------------+-----------+
| name         | state       | district  |
+--------------+-------------+-----------+
| Rahul Sharma | Maharashtra | Pune      |
| Neha Verma   | Delhi       | New Delhi |
| Amit Patil   | Maharashtra | Mumbai    |
+--------------+-------------+-----------+
3 rows in set (0.00 sec)
```

---

2. Left Join

```
mysql> SELECT u.name, d.doc_name
    -> FROM user u
    -> LEFT JOIN userdocument d ON u.user_id = d.user_id;
+--------------+------------------+
| name         | doc_name         |
+--------------+------------------+
| Rahul Sharma | Aadhaar Card     |
| Rahul Sharma | PAN Card         |
| Neha Verma   | Driving License  |
| Amit Patil   | NULL             |
+--------------+------------------+
4 rows in set (0.00 sec)
```

---

3. Right Join

```
mysql> SELECT c.full_name, u.phone_number
    -> FROM user u
    -> RIGHT JOIN citizenprofile c ON u.user_id = c.user_id;
+------------------+--------------+
| full_name        | phone_number |
```

```
+-------------------+--------------+
| Rahul S. Sharma   | 9876543210   |
| Neha Verma        | 8765432109   |
| Amit Patil        | 7654321098   |
+-------------------+--------------+
3 rows in set (0.00 sec)
```

4. Subquery

```
mysql> SELECT full_name, state
    -> FROM citizenprofile
    -> WHERE user_id IN
    -> (SELECT user_id FROM user WHERE language='Hindi');
+--------------+--------+
| full_name    | state  |
+--------------+--------+
| Neha Verma   | Delhi  |
+--------------+--------+
1 row in set (0.00 sec)
```

---

5. Nested Subquery

```
mysql> SELECT name, email
    -> FROM user
    -> WHERE user_id IN (
    -> SELECT user_id FROM userdocument
    -> WHERE doc_name = 'Aadhaar Card');
+--------------+------------------------+
| name         | email                  |
+--------------+------------------------+
| Rahul Sharma | rahul@email.com        |
+--------------+------------------------+
1 row in set (0.00 sec)
```

---

6. Create View

```
mysql> CREATE VIEW UserView AS
    -> SELECT u.user_id, u.name, c.state, c.district
    -> FROM user u
    -> JOIN citizenprofile c ON u.user_id = c.user_id;
Query OK, 0 rows affected (0.02 sec)
```

---

7. Query View

```
mysql> SELECT * FROM UserView
    -> WHERE state='Maharashtra';
+---------+--------------+-------------+----------+
```

```
| user_id | name         | state       | district |
+---------+--------------+-------------+----------+
| 1       | Rahul Sharma | Maharashtra | Pune     |
| 3       | Amit Patil   | Maharashtra | Mumbai   |
+---------+--------------+-------------+----------+
2 rows in set (0.00 sec)
```

Part B: GROUP BY and HAVING

Theory: GROUP BY and HAVING

GROUP BY is used to organize data into groups, while HAVING is used to apply conditions on grouped data. These are essential for data analysis and reporting.

8. GROUP BY

```
mysql> SELECT state, COUNT(profile_id) AS total_users
    -> FROM citizenprofile
    -> GROUP BY state;
+-------------+-------------+
| state       | total_users |
+-------------+-------------+
| Maharashtra | 2           |
| Delhi       | 1           |
+-------------+-------------+
2 rows in set (0.00 sec)
```

---

9. GROUP BY with HAVING

```
mysql> SELECT state, COUNT(profile_id) AS total_users
    -> FROM citizenprofile
    -> GROUP BY state
    -> HAVING COUNT(profile_id) > 1;
+-------------+-------------+
| state       | total_users |
+-------------+-------------+
| Maharashtra | 2           |
+-------------+-------------+
1 row in set (0.00 sec)
```

---

10. GROUP BY with JOIN

```
mysql> SELECT u.name, COUNT(d.doc_id) AS total_docs
    -> FROM user u
    -> LEFT JOIN userdocument d ON u.user_id = d.user_id
    -> GROUP BY u.name;
+--------------+------------+
| name         | total_docs |
+--------------+------------+
| Rahul Sharma | 2          |
| Neha Verma   | 1          |
| Amit Patil   | 0          |
+--------------+------------+
3 rows in set (0.00 sec)
```

Conclusion:

In this assignment, various SQL operations such as joins, subqueries, and views were successfully implemented. Additionally, GROUP BY and HAVING clauses were used to perform data grouping and filtering. These concepts are essential for efficient data retrieval and analysis in real-world database applications.